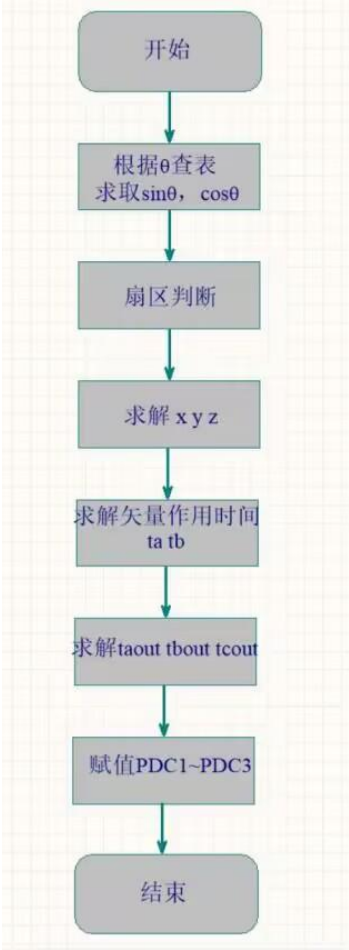
43-SVPWM 14

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

我们课程将要一步步走入深水区了，前面内容是后面的基础，大家需要跟紧不要掉队。我们开始上课啊，从今天这次课开始，我们就来分析下 SVPWM 控制部分的代码了。我们先从小的模块代码来分析，最后再把这些模块联系起来，构成一个工程。或者，我们先把每个小的模块代码分析清楚，再来给大家讲下整体的工程代码框架，最后到整体的工程功能实现。

之前呢，给大家分析了正弦查表模块的代码，大家应该都清楚了吧。那么，我们今天就主要来分析下 SVPWM 模块的代码实现。



根据之前的正弦查表函数， θ 对应的 sin θ 及 cos θ 已经可以获取到了。根据流程图呢，可以做一个扇区判断。这个扇区判断的方法，我们之前是讲过的，就是根据 θ 这个角度来进行判断。第Ⅰ扇区， θ 的范围是 0°~60°；第Ⅱ扇区， θ 的范围是 60°~120°。也就是每 60° 一个扇区，所以啊，只要 θ 是清楚的，自然这个电压矢量所处的扇区就是相对清楚的。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*============================================================================*/
static void  GetUsSector(unsigned int Angle)
{
    if((Angle >= 0) && (Angle < EANGLE60))
        tUm.Sector = 1;
    else if((Angle >= EANGLE60) && (Angle < EANGLE120))
        tUm.Sector = 2;
    else if((Angle >= EANGLE120) && (Angle < EANGLE180))
        tUm.Sector = 3;
    else if((Angle >= EANGLE180) && (Angle < EANGLE240))
        tUm.Sector = 4;
    else if((Angle >= EANGLE240) && (Angle < EANGLE300))
        tUm.Sector = 5;
    else
        tUm.Sector = 6;
}
```

此时大家看下这段代码，我们写了一个带形参的函数，这个形参是也就对应电压矢量的角度。 根据划分的原则呢，定义了一个 Sector 变量，来存储扇区号。判断的方法，就是检测 Angle 是在什么范围。而这里要注意的是，用的角度都是(0.16)格式的。

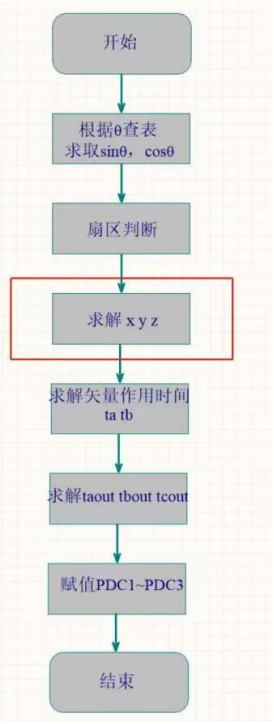
```
/*====================================
#define     EANGLE0     0
#define     EANGLE30    5461
#define     EANGLE60    10922
#define     EANGLE90    16384
#define     EANGLE120   21845
#define     EANGLE150   27306
#define     EANGLE180   32768
#define     EANGLE210   38229
#define     EANGLE240   43690
#define     EANGLE270   49151
#define     EANGLE300   54613
#define     EANGLE330   60074
#define     EANGLE360   0
```

这是这些角度的定义方式，也就是先人为的把 360° 角度范围内的角度先进行归一化。归一化之后呢，再乘以 65536，也就是转换为了(0.16)格式的角度值。当然了，这个转换过程呢，都是我们编程者自己先人为转换好的，程序里就是最终的值。这里需要说明的是，如果你把角度定义为 Q15，那么数值上和上面定义也是一样的。也就是说究竟这些常量是 Q16 还是 Q15，取决于程序员想把它当成什么什么格式使用。

问：在项目选择 Q15 和 Q16 有啥依据？

答：对于汇编而言，取决于程序员使用有符号指令还是无符号指令。对于 C 语言而言，取决于定义变量是无符号还是有符号。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



根据流程图，下一步是要求解 x，y，z

$$x = \sin\theta$$

$$y = \frac{1}{2}\sin\theta + \frac{\sqrt{3}}{2}\cos\theta$$

$$z = \frac{1}{2}\sin\theta - \frac{\sqrt{3}}{2}\cos\theta$$

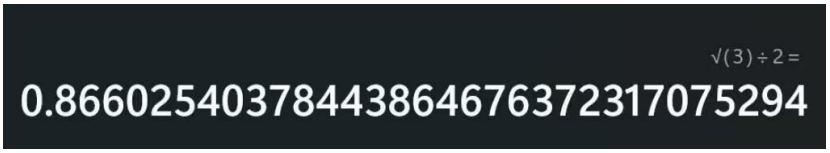
这个 x，y，z 的表达式是这样的。这个表达式中，有查表得到的 sin θ 及 cos θ ，也有常数 1/2 及 sqrt(3)/2 。这里呢，根据公式去求解 x，y，z 是相对简单的，我们只需要按照公式转换为相应的代码。

```
/*==========================================================================
三相相位相差60度的正弦波Ux Uy Uz产生
 Ux = Sin (θ)
 Uy = Sin (θ + 60)
    = 1/2 Sinθ + sqrt(3)/2 Cosθ
 Uz = Sin (θ - 60)
    = 1/2 Sinθ - sqrt(3)/2 Cosθ
==========================================================================*/
static void GetUxyz(void)
{
    UNION_S32 Temp32;

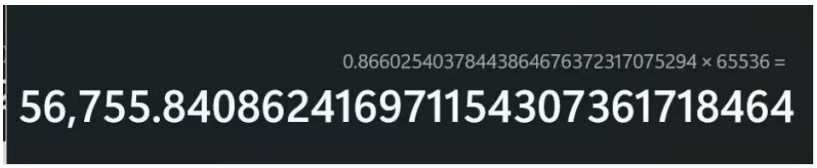
    tUm.Ux = tCore.qSin ;
    Temp32.S32 = __builtin_mulus(SQRT3DIV2,tCore.qCos);
    tUm.Uy = (tCore.qSin >> 1) + Temp32.Words.High;
    tUm.Uz = (tCore.qSin >> 1) - Temp32.Words.High;
}
```

此时，大家来看下这段代码。程序中呢，x，y，z 我使用了 Ux，Uy，Uz 来表示。Ux 呢，就直接使用查表得到的 sin θ 赋值得到。然后呢，我们先把 sqrt(3)/2 * cos θ 求解出来。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



sqrt(3)/2　转换为小数就是这个值，然后，我们把这个值转换为(0.16)格式的。也就是把这个小数，乘以 65536。



因为要取整数，则我们可以取为 56756。

```
/*==========================================
#define     SQRT3DIV2     56756
/*==========================================
```

这里呢，就用一个宏定义来表示 sqrt(3)/2。

```
/*============================================================
三相相位相差60度的正弦波Ux Uy Uz产生
 Ux = Sin (θ)
 Uy = Sin (θ + 60)
    = 1/2 Sinθ + sqrt(3)/2 Cosθ
 Uz = Sin (θ - 60)
    = 1/2 Sinθ - sqrt(3)/2 Cosθ
============================================================
static void GetUxyz(void)
{
    UNION_S32 Temp32;

    tUm.Ux = tCore.qSin ;
    Temp32.S32 = __builtin_mulus(SQRT3DIV2,tCore.qCos);
    tUm.Uy = (tCore.qSin >> 1) + Temp32.Words.High;
    tUm.Uz = (tCore.qSin >> 1) - Temp32.Words.High;
}
```

所以，这个函数中的乘法呢，也就对应一个(0.16)*　Q15 的乘法。那么，乘完之后呢，我们要把数据再转换为一个 Q15 格式的数据的话，就要取结果的高 16 位。

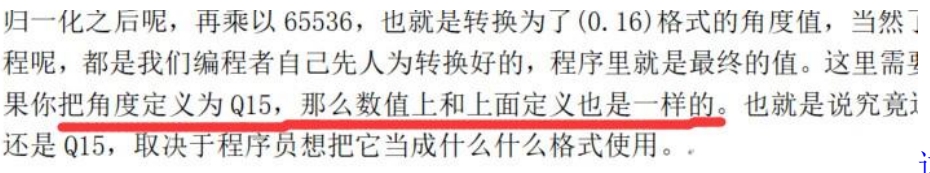
```
/*============================================================
三相相位相差60度的正弦波Ux Uy Uz产生
 Ux = Sin (θ)
 Uy = Sin (θ + 60)
    = 1/2 Sinθ + sqrt(3)/2 Cosθ
 Uz = Sin (θ - 60)
    = 1/2 Sinθ - sqrt(3)/2 Cosθ
============================================================
static void GetUxyz(void)
{
    UNION_S32 Temp32;

    tUm.Ux = tCore.qSin ;
    Temp32.S32 = __builtin_mulus(SQRT3DIV2,tCore.qCos);
    tUm.Uy = (tCore.qSin >> 1) + Temp32.Words.High;
    tUm.Uz = (tCore.qSin >> 1) - Temp32.Words.High;
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

那么，这样的话，Uy 跟 Uz 的代码实现，自然也就清楚了。1/2 * sinθ，可以使用右移一位来代替。所以，我们之前讲到的 Q 格式数据，以及它们之间的乘法，在程序中都用到了。这也就是之前跟大家说，所有人必须要掌握的原因了，后面程序中会有大量这样的用法。

前面内容，大家有疑问么？


归一化之后呢，再乘以 65536，也就是转换为了(0.16)格式的角度值，当然
程呢，都是我们编程者自己先人为转换好的，程序里就是最终的值。这里需
果你把角度定义为 Q15，那么数值上和上面定义也是一样的。也就是说究竟
还是 Q15，取决于程序员想把它当成什么什么格式使用。。

问：这个没听懂，啥意思呀？

答：比如 Q16 定义 330 度 #define EANGLE330 60074，也就是相当于 Q15 定义-30 度 #define _EANGLE30 60074。

问：用电压合成角判断的扇区，能不能用 Park 角判断扇区呢？这两个有什么异同？

答：不能，因为 SVPWM 的控制对象是 Us， 而 Park 角和 Us 的角度之间没有绝对的关系。扇区对针对电压矢量 Us 而言的。

问：在 C 语言中，有符号数和无符号数 强制转换的时候，值有变化么？

答：如果是正数的话，没有影响，是负数的话就不对了。

问：有的地方，有符号强制变成无符号，值是 2 倍了是什么意思呢？

答：比如 Q15 的 0.5 = 16384 Q16 的 0.5=32768 你看 Q15 的 0.5 强制转换到 Q16 的时候，需要乘 2 吧。

问：有符号的 强制转换无符号，就相当于除以 2 了，是这样的吧？

答：Q15 的 0.5 强制转换到 Q16 的时候，需要乘 2； Q16 的 0.5 强制转换到 Q15 的时候，需要除以 2。

问：这个编程的时候就的很小心了，一旦转换就必须运算，但是只有这款单片机，小数格式的，才需要这样计算么？

答：采用 Q 格式运算的其它单片机也是这样处理的。

求解 x，y，z 的目的呢，是要求解矢量的作用时间

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

$$x = \sin\theta$$
$$y = -\frac{1}{2}\sin\theta + \frac{\sqrt{3}}{2}\cos\theta$$
$$z = -\frac{1}{2}\sin\theta - \frac{\sqrt{3}}{2}\cos\theta$$

扇区I	扇区II	扇区III
$T1 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-z)$ $T6 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}x$ $T0 = T - T6 - T1$	$T6 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(y)$ $T5 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(z)$ $T0 = T - T5 - T6$	$T5 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(x)$ $T4 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-y)$ $T0 = T - T4 - T5$
扇区IV	**扇区V**	**扇区VI**
$T4 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(z)$ $T3 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-x)$ $T0 = T - T3 - T4$	$T3 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-y)$ $T2 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-z)$ $T0 = T - T2 - T3$	$T2 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(-x)$ $T1 = \frac{2*T*\lvert Us\rvert}{\sqrt{3}*Udc}(y)$ $T0 = T - T1 - T2$

而矢量的作用时间呢，我们之前讲过，六个扇区对应的时间呢是不同的。而求解矢量作用时间呢，最终是为了把 PWM 模块的比较值寄存器的值给确定下来。

$$taout = \frac{ta}{2} + tbout$$
$$tbout = \frac{tb}{2} + tcout$$
$$tcout = \frac{(\frac{T}{2} - \frac{ta}{2} - \frac{tb}{2})}{2}$$

第六扇区	第五扇区	第四扇区	第三扇区	第二扇区	第一扇区
PDC1 = taout	PDC1 = tbout	PDC1 = tcout	PDC1 = tcout	PDC1 = tbout	PDC1 = taout
PDC2 = tbout	PDC2 = taout	PDC2 = taout	PDC2 = tbout	PDC2 = tcout	PDC2 = tcout
PDC3 = tcout	PDC3 = tcout	PDC3 = tbout	PDC3 = taout	PDC3 = taout	PDC3 = tbout

所以呢，我们就推导得出了这样一个汇总的表。但是呢，有 6 个扇区对应的值要去求解，我们也不想 6 个扇区都求解一遍。所以啊，我们编程的时候呢，可以写一个公共的函数。

$$taout = \frac{ta}{2} + tbout$$
$$tbout = \frac{tb}{2} + tcout$$
$$tcout = \frac{(\frac{T}{2} - \frac{ta}{2} - \frac{tb}{2})}{2}$$

第六扇区	第五扇区	第四扇区	第三扇区	第二扇区	第一扇区
PDC1 = taout	PDC1 = tbout	PDC1 = tcout	PDC1 = tcout	PDC1 = tbout	PDC1 = taout
PDC2 = tbout	PDC2 = taout	PDC2 = taout	PDC2 = tbout	PDC2 = tcout	PDC2 = tcout
PDC3 = tcout	PDC3 = tcout	PDC3 = tbout	PDC3 = taout	PDC3 = taout	PDC3 = tbout

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个函数的目的呢，是先求解出公共的 taout，tbout，tcout。然后，再根据求解出来的扇区号，来进行分配赋值 PDC1~PDC3。所以呢，下面我们就先来看下这个公共函数是怎么实现的？实际上，这个公共函数中，需要有两个输入量，ta 跟 tb，不同的扇区呢，这个 ta 跟 tb 是不一样的。

```
/*=============================================================================*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

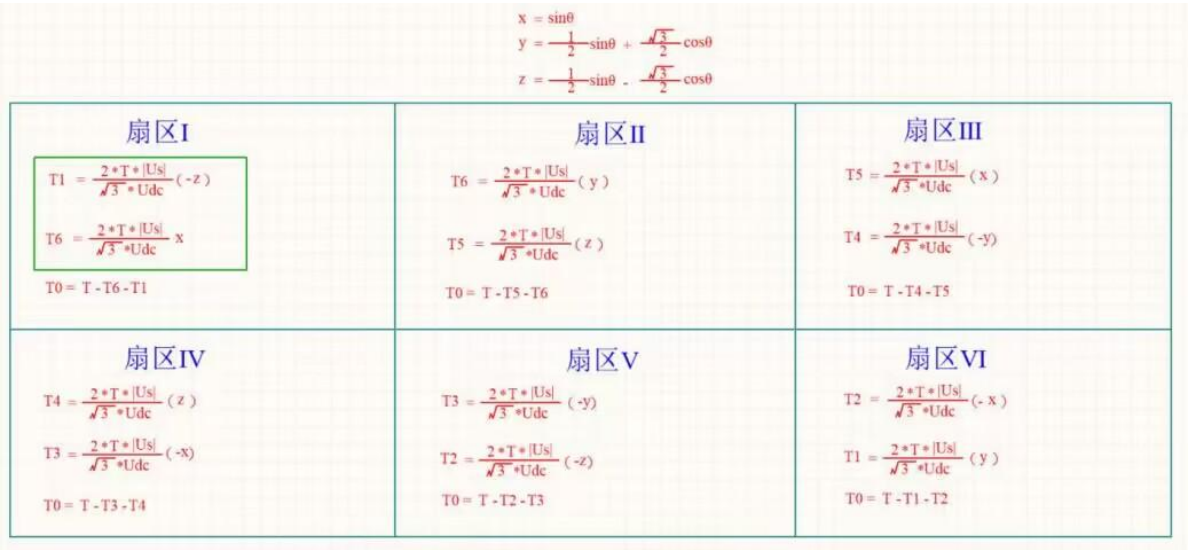
    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
```

此时，我们来看下这个函数，函数带有两个形参，这两个形参，就对应我们刚才讲的 ta，tb。



实际上，ta，tb 分别对应每个扇区的第一矢量作用时间，以及第二矢量作用时间。但是呢，因为不同扇区中，前面的分式是一个公共的部分。所以，公共部分的代码实现呢，就放到了函数内部。所以啊，输入的形参呢，Sa，Sb，其实就对应 不同扇区的 x y z。例如，第一扇区是 Sa = -z，Sb = x。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=====================================================================================*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

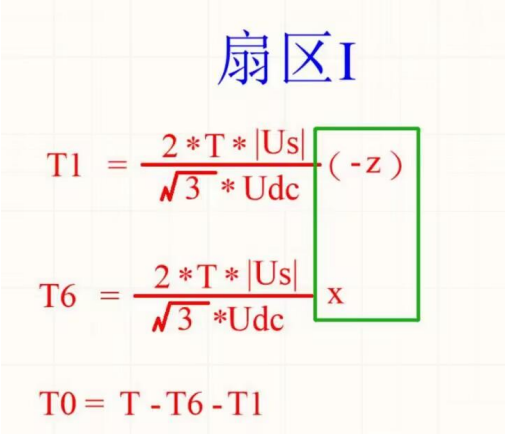
    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

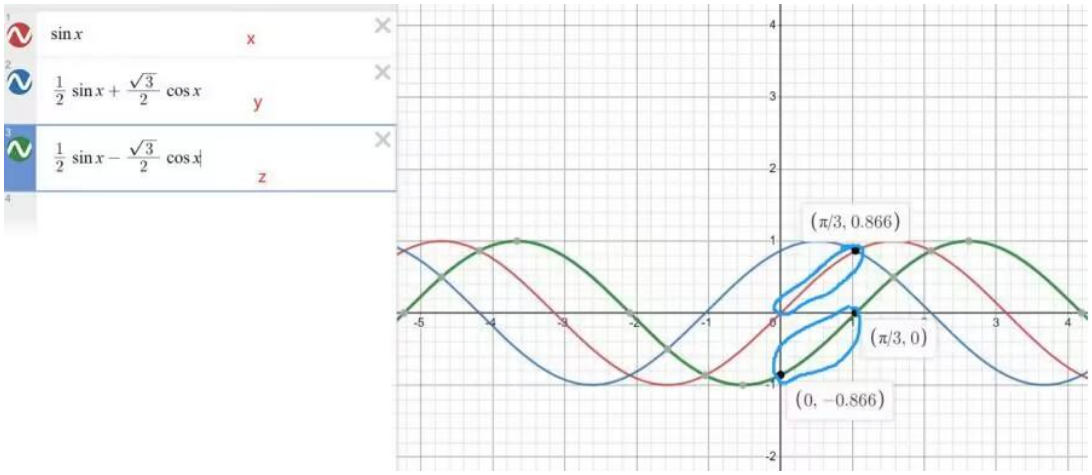
    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;
```

函数对应的两个形参，就对应不同扇区的 x y z 的组合。那么，这里为什么代码中还要判断 Sa，Sb 小于 0，然后限制为 0 呢？ 也就是，只能确保 Sa，Sb 是大于等于 0 的数，不能出现负数，这里为啥这样做呢？



大家看下，第一扇区对应的是 -z 跟 x



我们可以把 x，y，z 的图象画出来。然后，第一扇区的话，角度是 0~60° 。红色 x 的话，从图象上来看是大于等于 0 的。而绿色 z 的话，是小于等于 0 的。所以，-z 的话，按理是大于等于 0 的。那么，这样的话，我们还有必要对-z 跟 x 做小于 0 的判断吗？

实际上，大家都知道，我们正弦值，余弦值是查表+线性插值得到的。然后呢，x y z 中又会涉及到加减操作。如果说，我们查表的值出现一点误差，或者运算中出现一点小数误

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

差，加减操作中得到的值不是 0，而是接近于 0 的负数。而我们程序中呢，从理论分析这些结果都是大于 0 的，所以把这些数据转换为无符号的数据来使用。如果我们没有小于 0 的判断，那么接近于 0 的很小的负数，转换为无符号之后，就会变为一个很大的正数。这样的话，就会导致计算结果出错。所以啊，为了防止这种情况的出现，我们对形参进行了判断，确保把这种会出现接近于 0 的负数的情况给避免掉。这样后续在计算的时候呢，也就不会有异常的数据出现，结果也就正确了。

```
/*=============================================================================*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

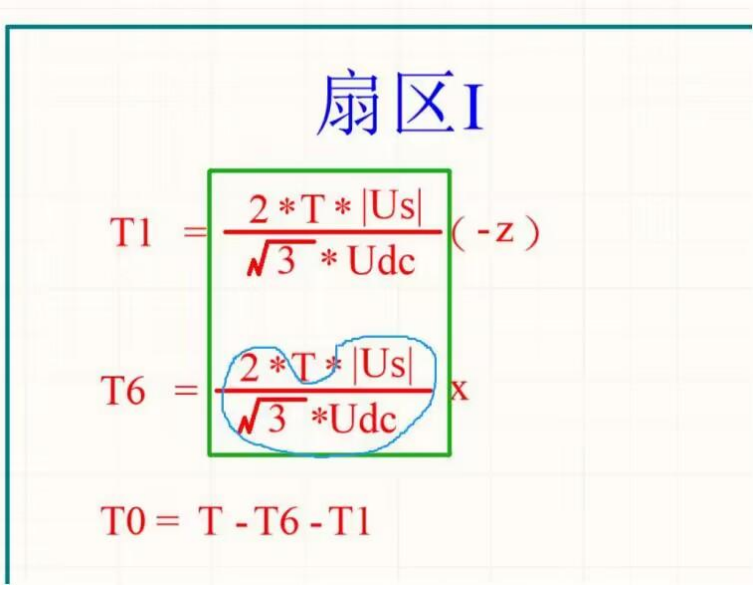
    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
```

所以，代码中，先对输入的两个形参进行判断，就是为了解决遇到输入的数据是接近于 0 的负的数据的情况。



接着呢，我们要用前面的公因式跟输入的 -z 或 x 进行相乘，而之前我们定义过，蓝圈部分使用 M 来表示，这样的话，矢量作用时间的表达式，其实可以写为

T1 = M * T * (-z)

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

T6 = M * T * x

而这个 M 呢，因为这个|Us|是在 0~ sqrt(3)/2 *Udc 之间变化，所以 M 就在 0~1 之间变化。在 SVPWM 控制这个部分呢，M 这个值呢，是我们人为的来给定，也就相当于人为的来调整电压合成矢量的模长|Us|。所以，这个蓝圈部分，我们程序中，可以定义一个 16 位无符号的变量来表示，归一化之后呢，它的值是在 0~1 之间，所以使用(0.16)格式的数据来表示，范围就是 0~65535 。

```
/*=============================================================================*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
```

代码中呢，tUm.Percent 这个变量，就用来表示(0.16)格式的 M。而红框中的这段代码呢，是用于限制这个 M 的最大值。一般呢，这个 M 也达不到 1，正常情况下是在 0.92~0.96 左右。所以，这里就有一个限幅的判断，目的就是将 M 限制在 0~0.92 这个范围内。

问：有感 M 可以做到 1 吗？
答：做不到，主要原因是要考虑预留采样电流窗口。

```
/*=============================================================================*/
static void  SvpwmInitialize(void)
{
    tUm.PercentMax = 60293;
}
/*=============================================================================*/
```

这个限幅值呢，是在程序初始化的时候，设定的一个值。60293 / 65536 = 0.92。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=============================================================================*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
```

而这段代码呢，就是来求解 M*T * (-z)以及 M*T*x（以第一扇区为例）。这里的 T 呢，我们载频是 20KHz，周期就是 50us，而代码中的 T 呢，已经转换为 PWM 模块的周期值。因为我们后面是需要对 PDCx 寄存器赋值呢。所以，这里直接就把时间 T，转换为 PWM 模块的周期值。

```
/*=============================================================================*/
#define     CARRIER_FREQUENCY    20000 //PWM载波频率
#define     PWM_PERIOD_VALUE     (120000000 / CARRIER_FREQUENCY)
/*=============================================================================*/
```

这个 T 呢，在程序中是用一个宏定义来定义的，计算下来呢，其实就是 6000。

```
/*=============================================================================*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
```

这个地方呢，大家可以理解为两个(0.16)格式的数据相乘，然后，要得到(0.16)格式的数据的话，要取结果的高 16 位。M 在 0~0.92 范围内的话，这个高 16 位的数据范围就是在 0~5520 范围内。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*-----------------------------------------------------------------------------*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

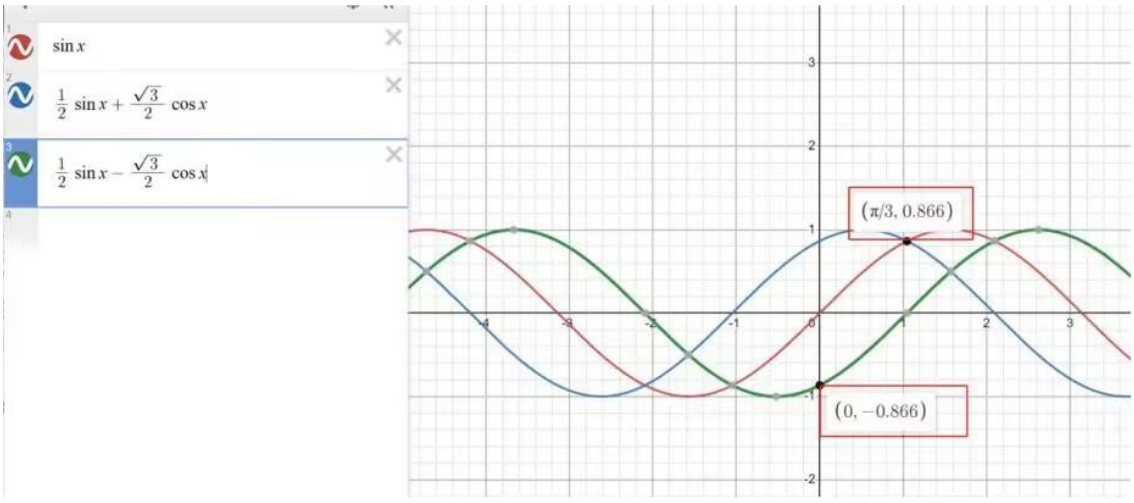
    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
```

也就是说，M * T 的数据范围，在程序中就是 0~5520。接着呢，用 0~5520 范围的数据跟 a 和 b 进行相乘。也就是要计算 M * T * (-z) 跟 M * T * (x)(以第一扇区为例)。这里的 a b 呢，经过之前的处理，已经变为了大于等于 0 的数据。并且呢，数据范围是在 0~32767 内，而因为一个扇区是 60°，所以这个数据范围还可以精确到 0~28378 范围内 （0~sqrt(3)/2）。



大家看这幅图就知道，在扇区的边界点，纵坐标的值是 0.866，也就对应着 sqrt(3)/2。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*=============================================================================*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T  0~5520
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2 a:0~28378  (0~sqrt(3)/2)
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2 b:0~28378  (0~sqrt(3)/2)

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
```

ab 值的范围确定了，那么乘积的范围就可以确定了。a　b 的话，原始的数据其实是 Q15，只不过呢，经过处理之后呢，变为了没有负数的部分。没有负数的话，又使用 16 位无符号变量来存储，所以呢，这里把它当成是　(0.16)格式的数据。而 Q15 的格式，跟(0.16)格式的数据来看，正数部分，它们之间有 2 倍的关系。Q15 的原始数据，转换为(0.16)格式的话，可以等效为数据缩小了 2 倍。所以啊，红框中的这一步，我们就相当于直接得到了　ta/2 和 tb/2　。

问：举个列子呗，这点理解不了？

答：比如 Q15 的 0.5，　16384 直接转换为 Q16，16384 就是小数 0.25 了。相当于小数缩小了 2 倍。

```
/*=============================================================================*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T  0~5520
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2 a:0~28378  (0~sqrt(3)/2)
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2 b:0~28378  (0~sqrt(3)/2)

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
```

也就是，这两个 32 位的高 16 位，就相当于是 ta/2 和 tb/2。这样的话，第一矢量作用时间，跟第二矢量作用时间，我们就求解出来了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

$$taout = \frac{ta}{2} + tbout$$

$$tbout = \frac{tb}{2} + tcout$$

$$tcout = \frac{(\frac{T}{2} - \frac{ta}{2} - \frac{tb}{2})}{2}$$

第六扇区	第五扇区	第四扇区	第三扇区	第二扇区	第一扇区
PDC1 = taout	PDC1 = tbout	PDC1 = tcout	PDC1 = tcout	PDC1 = tbout	PDC1 = taout
PDC2 = tbout	PDC2 = taout	PDC2 = taout	PDC2 = tbout	PDC2 = tcout	PDC2 = tcout
PDC3 = tcout	PDC3 = tcout	PDC3 = tbout	PDC3 = taout	PDC3 = taout	PDC3 = tbout

求解出来之后呢，也就可以继续求解 taout tbout tcout 了。

```
/*-----------------------------------------------------------------------------*/
static void VectorActionTimeCalculate(signed int Sa,signed int Sb)
{
    UNION_U32 Temp32;
    UNION_U32 Temp32A,Temp32B;

    unsigned int a,b;

    if(Sa<0)
        Sa = 0;
    if(Sb<0)
        Sb = 0;

    a = (unsigned int)Sa;
    b = (unsigned int)Sb;

    if(tUm.Percent > tUm.PercentMax)
        tUm.Percent = tUm.PercentMax;

    Temp32.U32 = __builtin_muluu(tUm.Percent,PWM_PERIOD_VALUE); //M * T  0~5520
    Temp32A.U32 = __builtin_muluu(Temp32.Words.High,a);//ta/2 a:0~28378  (0~sqrt(3)/2)
    Temp32B.U32 = __builtin_muluu(Temp32.Words.High,b);//tb/2 b:0~28378  (0~sqrt(3)/2)

    tSvpwm.Tcout = ((PWM_PERIOD_VALUE >> 1) - Temp32A.Words.High - Temp32B.Words.High) >> 1;
    tSvpwm.Tbout = tSvpwm.Tcout + Temp32B.Words.High;
    tSvpwm.Taout = tSvpwm.Tbout + Temp32A.Words.High;
}
/*-----------------------------------------------------------------------------*/
```

我们今天就讲到这里吧，下面回答大家的问题。

问：请问 T 的求解过程中的 120000000 是怎么确定的？
答：fp = 120MHz，PWM 模块的输入频率是 120MHz。

问：咱们那个选 0.92 和 0.9 有什么不同吗？就是圆半径不同吗？
答：是的，圆的半径不同。主要看电流采样窗口需要多大，比如 ADC 采样时间要多长，硬件开关振荡时间有多长？

问：这个宏定义数据计算后为 6000，代表周期 50us。为什么不用 50us 参与计算二是使用 6000？
答：因为我们最终希望得到的是比较值，这个比较值就相当于时间， 和时间有对应关系。

大家早点休息，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。